



University of Asia Pacific

Department of Computer Science and Engineering
CSE 450: VLSI Lab

LAB REPORT

Experiment Number: 04

Experiment Title: Design and Simulation of an 8-bit ALU

Submitted by:

Name: Naima Tabassum Antora

Student ID: 22201068

Section: C1

Submitted to:

Md. Rasheduzzaman

Lecturer

Department of Computer Science and Engineering

Date of Submission: 3 October, 2026

1. Experiment Name

Design and Simulation of an 8-bit ALU

2. Objective

- To design and simulate an 8-bit Arithmetic Logic Unit (ALU) using Verilog HDL.
- To implement basic ALU operations such as AND, OR, XOR, NOT, addition, and subtraction.
- To build the 8-bit ALU by combining multiple 1-bit ALU blocks.
- To use opcode inputs to select different arithmetic and logical operations.
- To verify the ALU outputs through RTL schematic and simulation in Xilinx ISE.

3. Apparatus / Hardware & Software Requirements

- **Xilinx ISE 14.7** for writing, synthesizing, and checking the Verilog design.
- **Verilog HDL** for implementing the 1-bit and 8-bit ALU modules.
- **ISim Simulator** for testing the ALU operations and observing the simulation waveforms.
- **Oracle VirtualBox** for running the Xilinx development environment.

4. Circuit Diagram / Schematic

The ALU design was developed step by step. First, the basic 1-bit operations were designed separately and their RTL schematics were checked. After that, the individual operations were combined into a single 1-bit ALU using opcode inputs. Finally, eight 1-bit ALU blocks were connected to form the complete 8-bit ALU.

Figure 1 — 1-bit AND

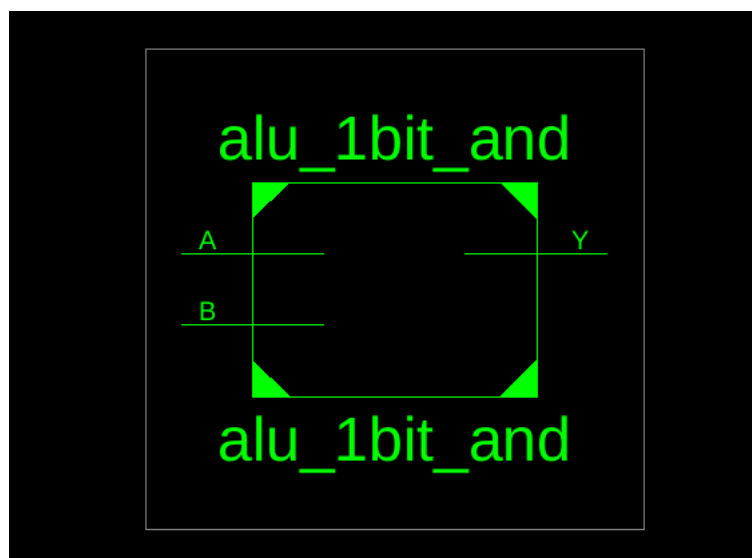


Figure 1(a): RTL Schematic of 1-bit AND Operation (Top level)

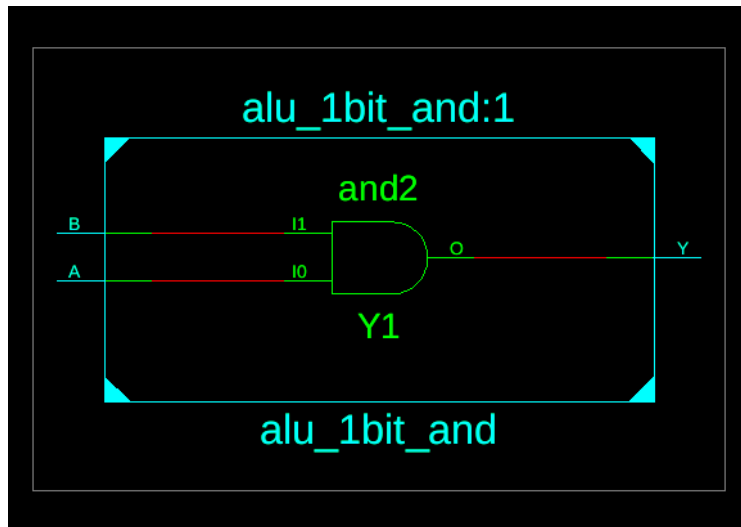


Figure 1(b): RTL Schematic of 1-bit AND Operation (Expanded)

The 1-bit AND block takes two inputs, A and B, and produces one output. The output becomes 1 only when both input values are 1. This block was used as one of the logical operations of the ALU.

Figure 2 — 1-bit OR

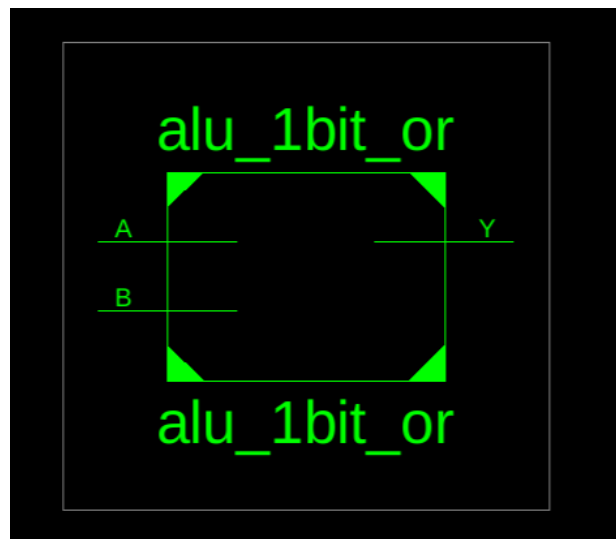


Figure 2(a): RTL Schematic of 1-bit OR Operation (Top level)

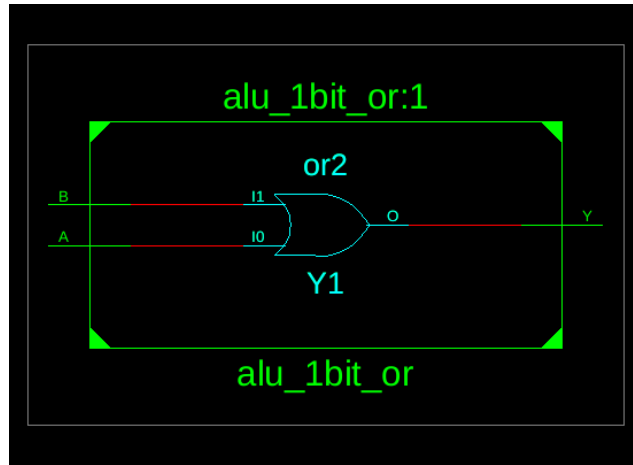


Figure 2(b): RTL Schematic of 1-bit OR Operation (Expanded)

The OR block also uses two inputs, A and B. The output becomes 1 when at least one of the inputs is 1. This operation was included as another basic logical function of the ALU.

Figure 3 — 1-bit XOR

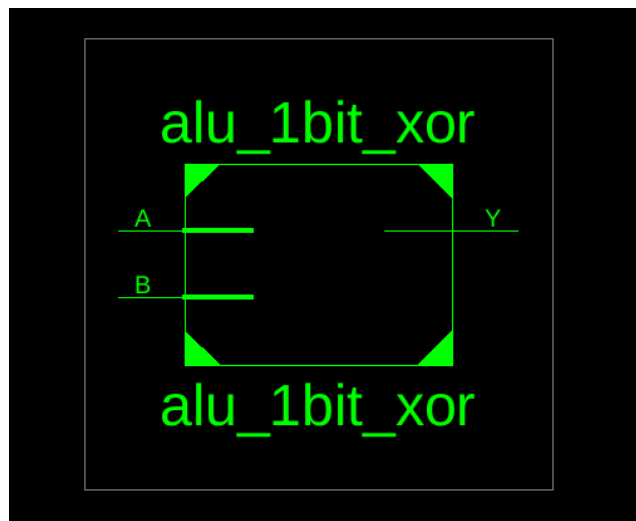


Figure 3(a): RTL Schematic of 1-bit XOR Operation (Top level)

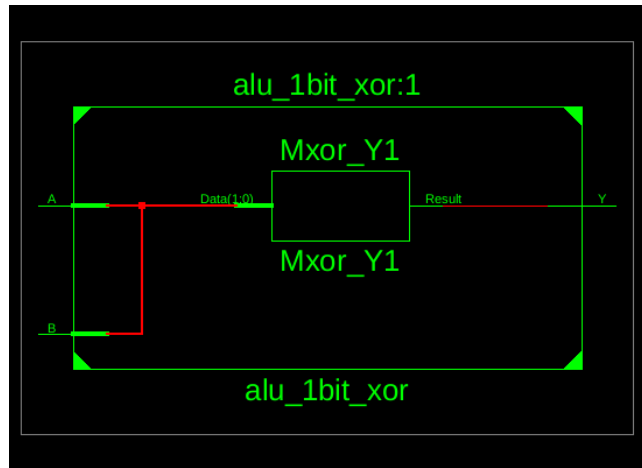


Figure 3(b): RTL Schematic of 1-bit XOR Operation (Expanded)

The XOR block compares A and B and gives an output of 1 when the two input values are different. The RTL schematic was checked to confirm the implementation of this logical operation.

Figure 4 — 1-bit NOT

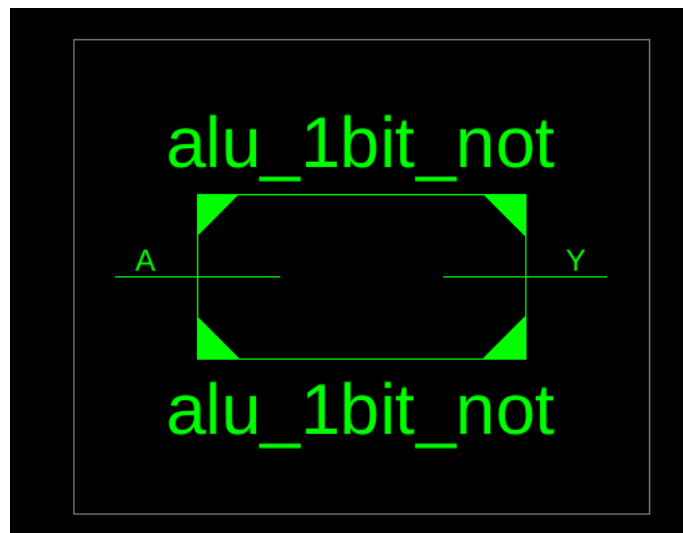


Figure 4(a): RTL Schematic of 1-bit NOT Operation (Top level)

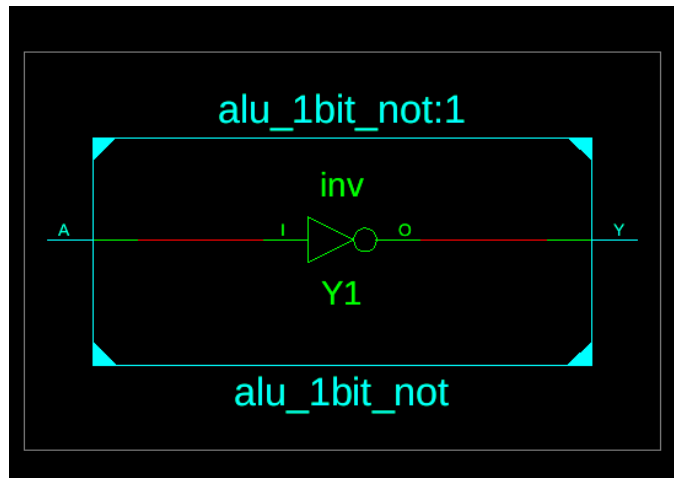


Figure 4(b): RTL Schematic of 1-bit NOT Operation (Expanded)

The NOT operation uses only input A and produces its opposite value. When A is 0, the output becomes 1, and when A is 1, the output becomes 0.

Figure 5 — 1-bit ADD

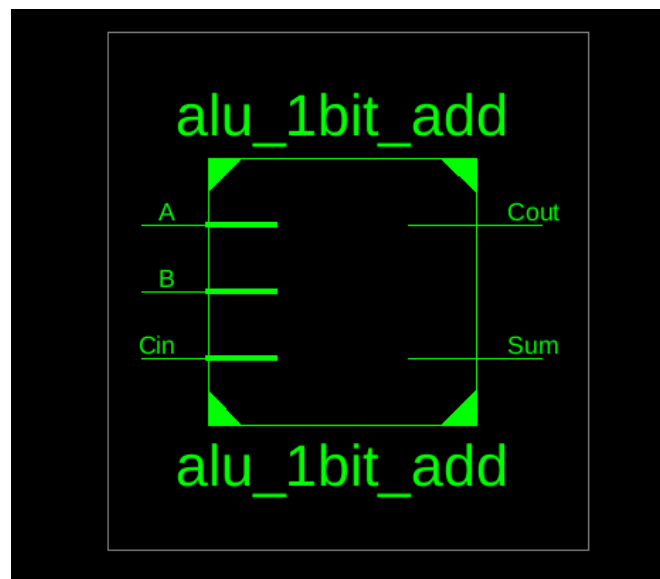


Figure 5(a): RTL Schematic of 1-bit Addition (Top level)

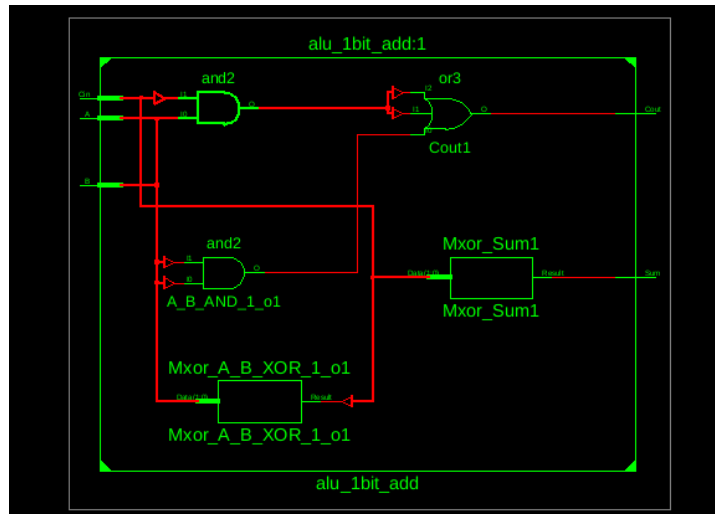


Figure 5(b): RTL Schematic of 1-bit Addition (Expanded)

The 1-bit addition block was designed using A, B, and Cin as inputs. It produces Sum and Cout as outputs. The carry output is used when the addition of the input bits produces a carry to the next bit.

Figure 6 — 1-bit SUBTRACT

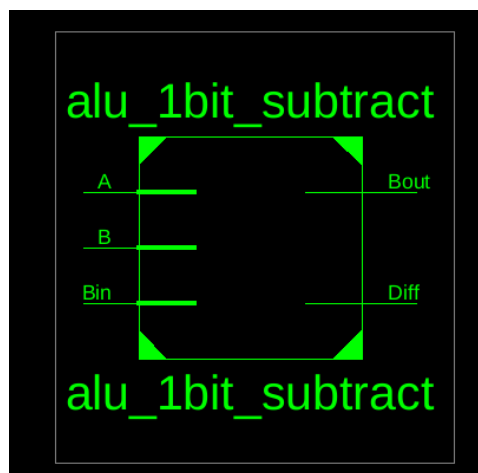


Figure 6(a): RTL Schematic of 1-bit Subtraction (Top level)

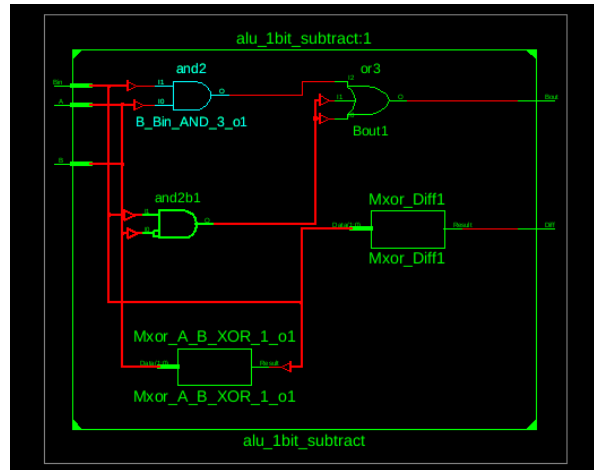


Figure 6(b): RTL Schematic of 1-bit Subtraction (Expanded)

The 1-bit subtraction block uses A, B, and Bin as inputs. It produces Diff and Bout as outputs. The borrow output is passed to the next bit when a borrow is required during subtraction.

Figure 7 — Combined 1-bit ALU

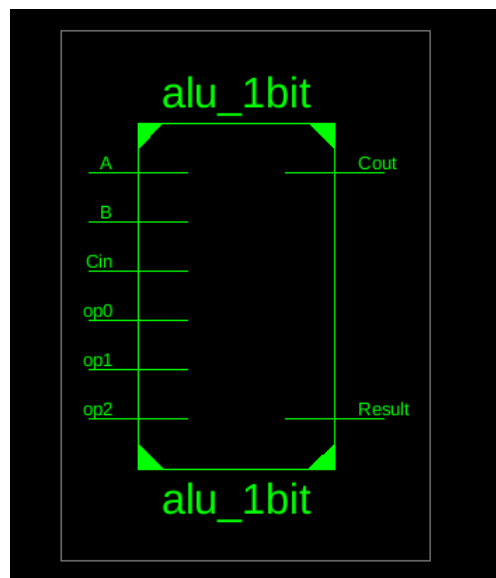


Figure 7(a): RTL Schematic of the Combined 1-bit ALU (Top level)

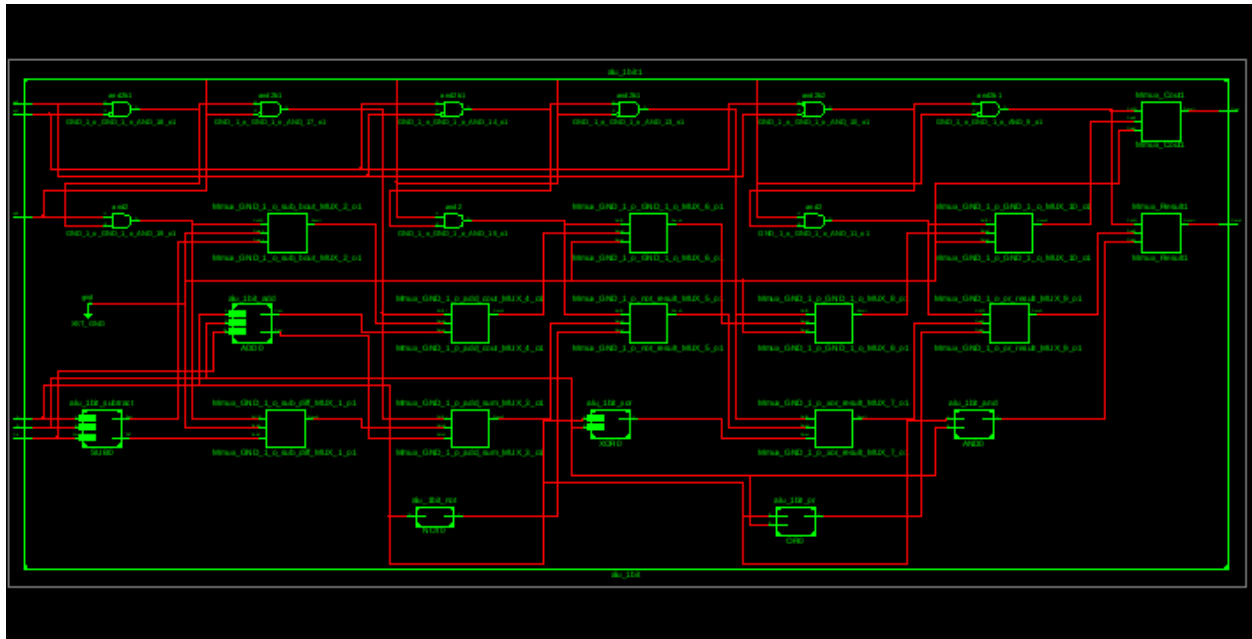


Figure 7(b): RTL Schematic of the Combined 1-bit ALU (Expanded)

After testing the individual operations, they were combined into one 1-bit ALU. The inputs A, B, and Cin are used for the data, while op2, op1, and op0 are used to select the required operation. The selected output is provided through Result, while Cout is used for carry or borrow during arithmetic operations.

Figure 8 — 8-bit ALU

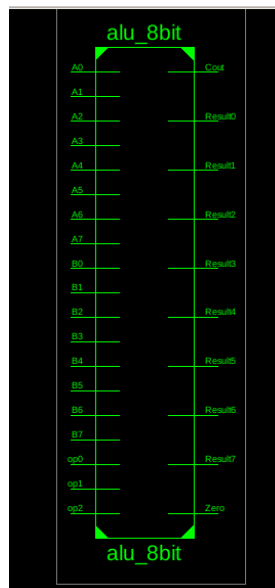


Figure 8(a): RTL Schematic of the 8-bit ALU (Top level)

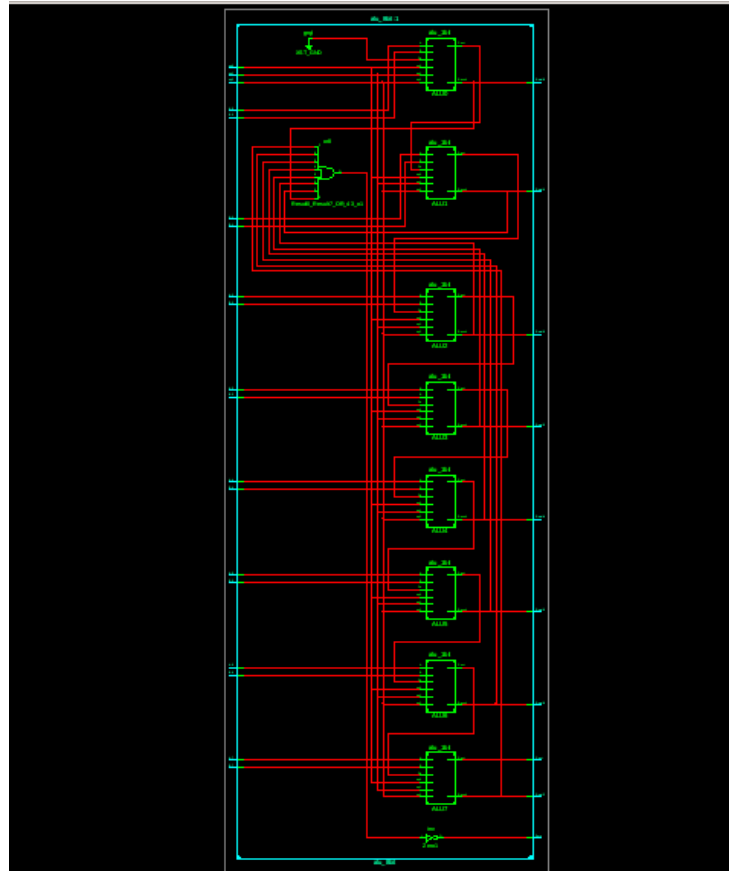


Figure 8(b): RTL Schematic of the 8-bit ALU (Expanded)

The final 8-bit ALU uses separate input bits A0 to A7 and B0 to B7. The opcode inputs are shared by all 1-bit ALU blocks so that the same operation is performed on all eight bits. The design produces Result0 to Result7, along with Cout and Zero outputs.

6. Output Analysis / Observations

The simulation results were observed in ISim to check whether each ALU operation produced the expected output. The individual 1-bit operations were tested first, followed by the combined 1-bit ALU and the final 8-bit ALU. The observed outputs matched the expected logical and arithmetic results for the applied test inputs.

1. 1-bit AND Simulation

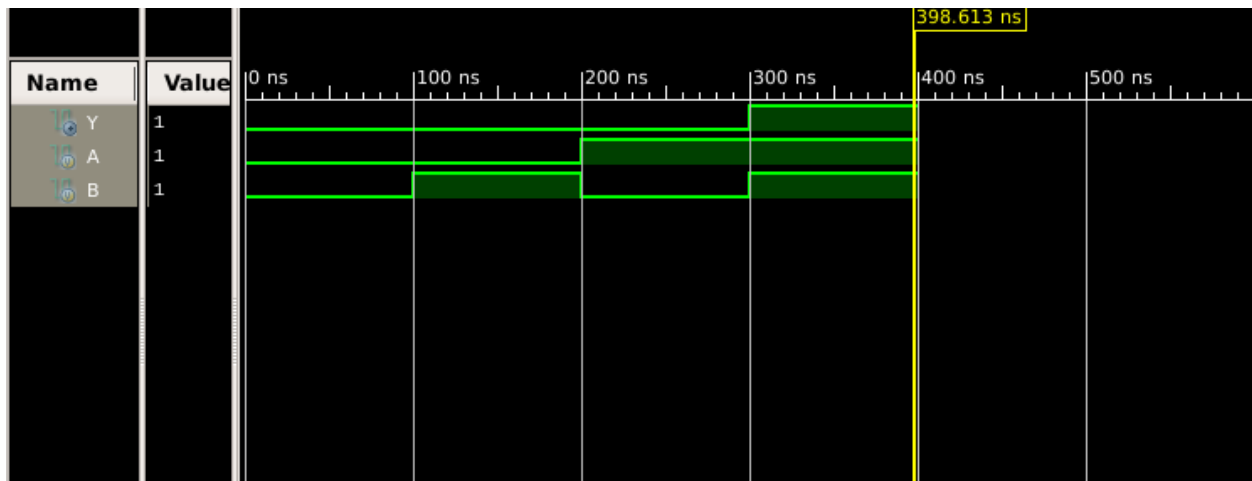


Figure 9: Simulation Waveform of 1-bit AND Operation

1-bit AND Operation Table:

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Table: Observation of 1-bit AND Operation

2. 1-bit OR Simulation

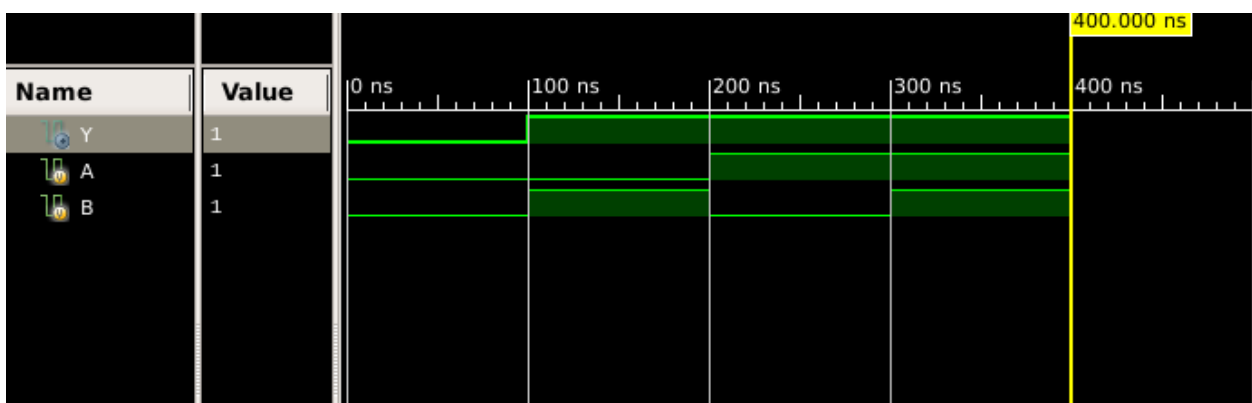


Figure 10: Simulation Waveform of 1-bit OR Operation

1-bit OR Operation Table:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Table: Observation of 1-bit OR Operation

31-bit XOR Simulation

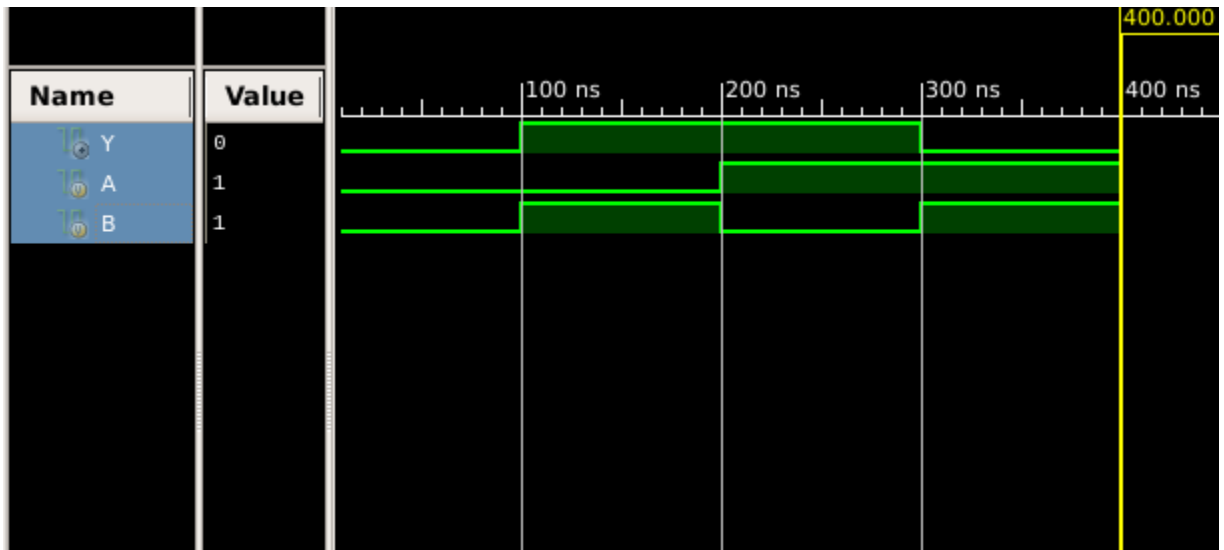


Figure 11: Simulation Waveform of 1-bit XOR Operation

1-bit XOR Operation Table:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

Table: Observation of 1-bit XOR Operation

4. 1-bit NOT Simulation

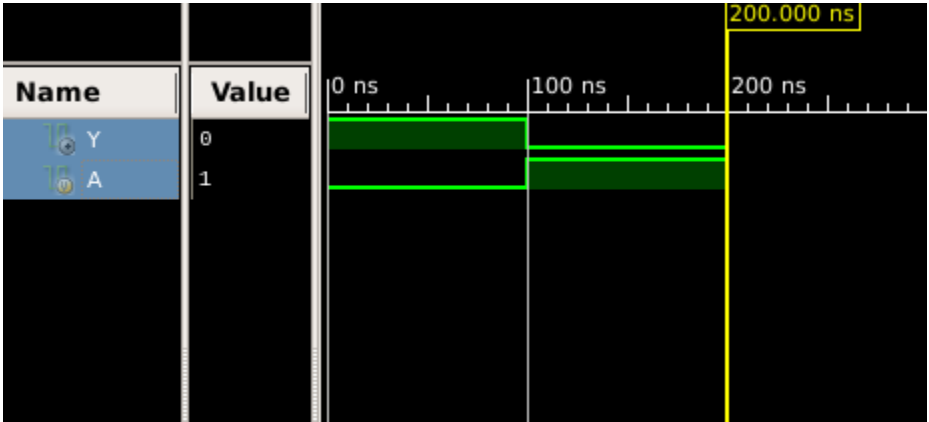


Figure 12: Simulation Waveform of 1-bit NOT Operation

1-bit NOT Operation Table:

A	Y
0	1
1	0

Table: Observation of 1-bit NOT Operation

5. 1-bit ADD Simulation

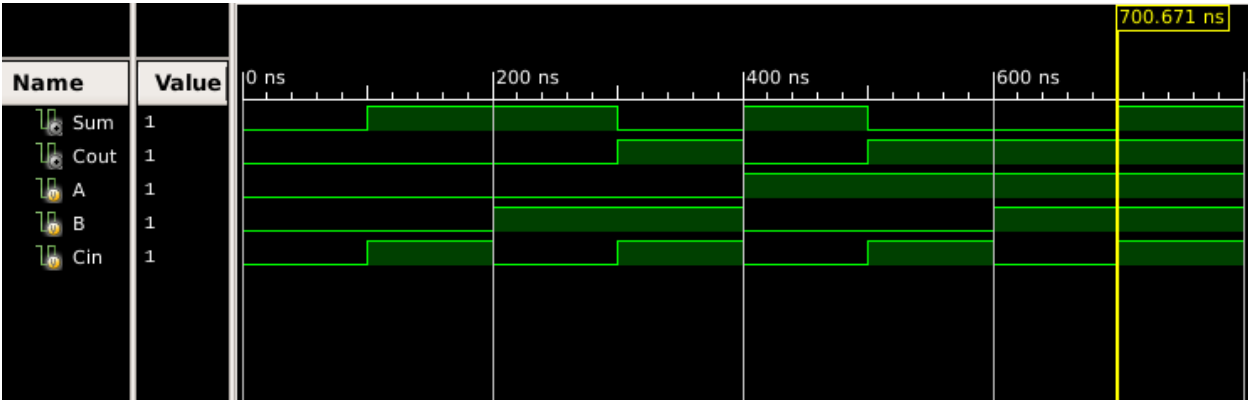


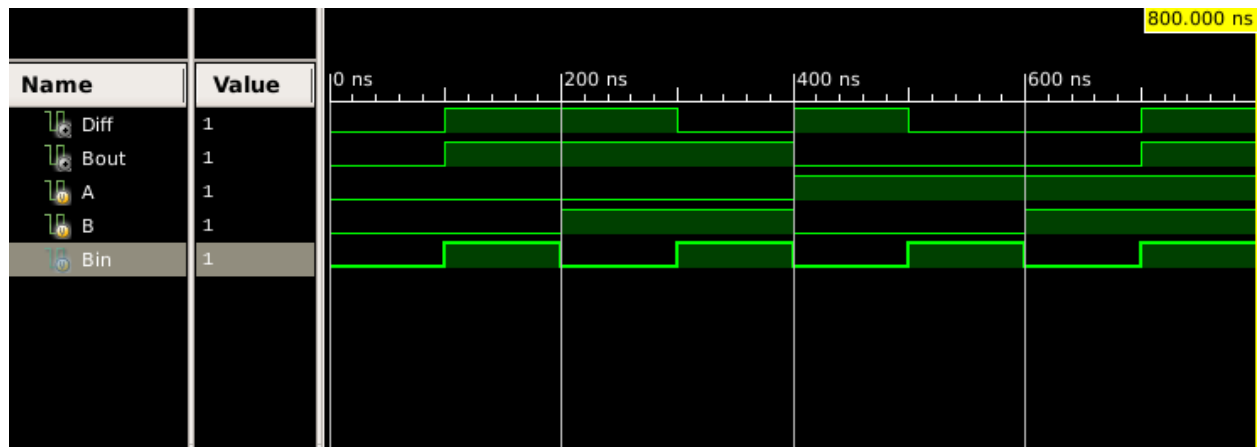
Figure 13: Simulation Waveform of 1-bit Addition

1-bit ADD Operation Table:

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table: Observation of 1-bit ADD Operation

6. 1-bit SUBTRACT Simulation



1-bit SUBTRACT Operation Table:

A	B	Bin	Diff	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1

A	B	Bin	Diff	Bout
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Table: Observation of 1-bit SUBTRACT Operation

7. Combined 1-bit ALU Simulation



Figure 15: Simulation Waveform of the Combined 1-bit ALU

Combined 1-bit ALU Table:

op2	op1	op0	Operation	A	B	Cin	Result	Cout
0	0	0	AND	1	1	0	1	0

op2	op1	op0	Operation	A	B	Cin	Result	Cout
0	0	1	OR	0	1	0	1	0
0	1	0	XOR	1	0	0	1	0
0	1	1	NOT	0	0	0	1	0
1	0	0	ADD	1	1	0	0	1
1	0	1	SUBTRACT	0	1	0	1	1

Table: Observation of Combined 1-bit ALU

8. Final 8-bit ALU Simulation

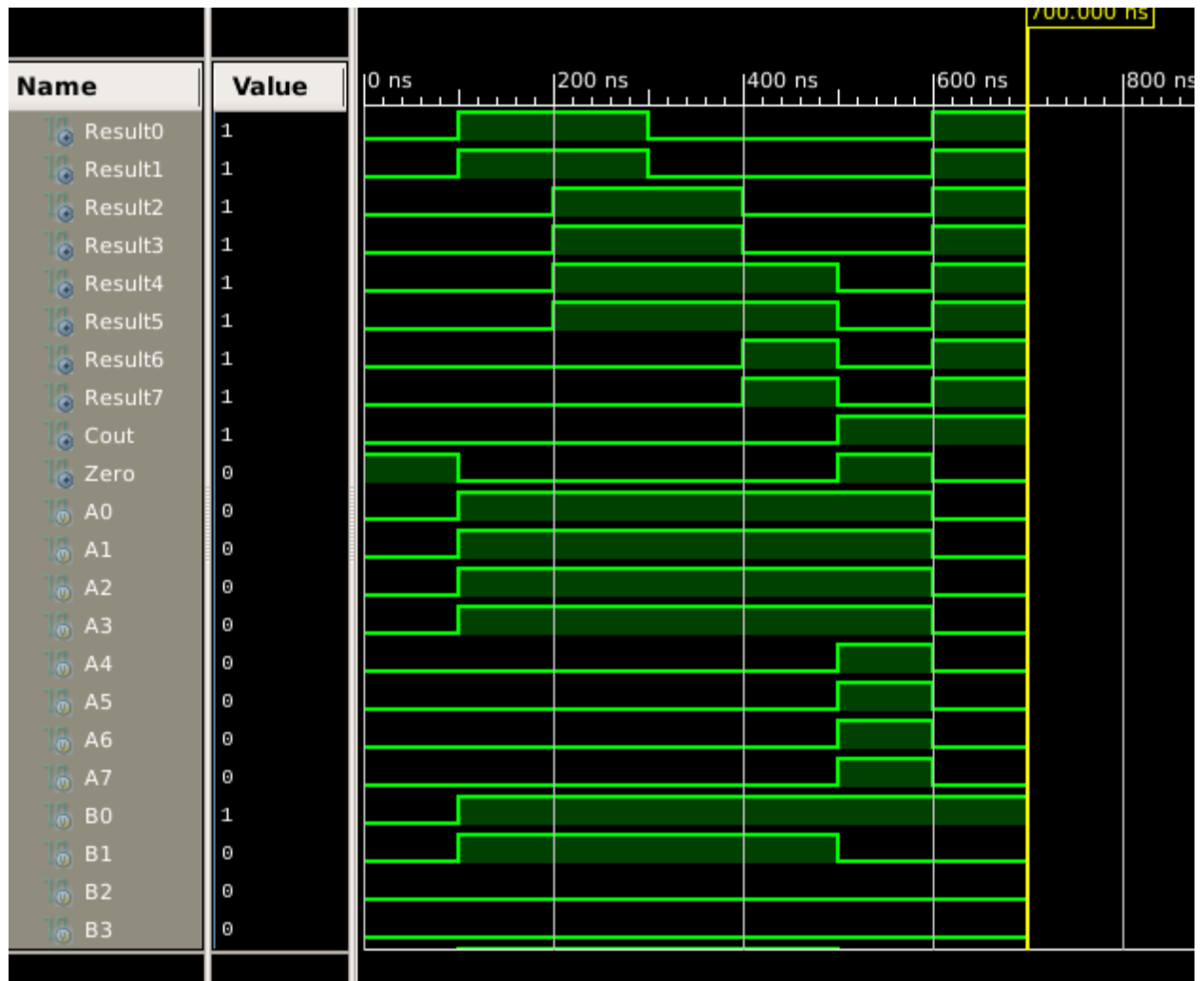


Figure 16(a): Simulation Waveform of the 8-bit ALU – Result and Output Signals

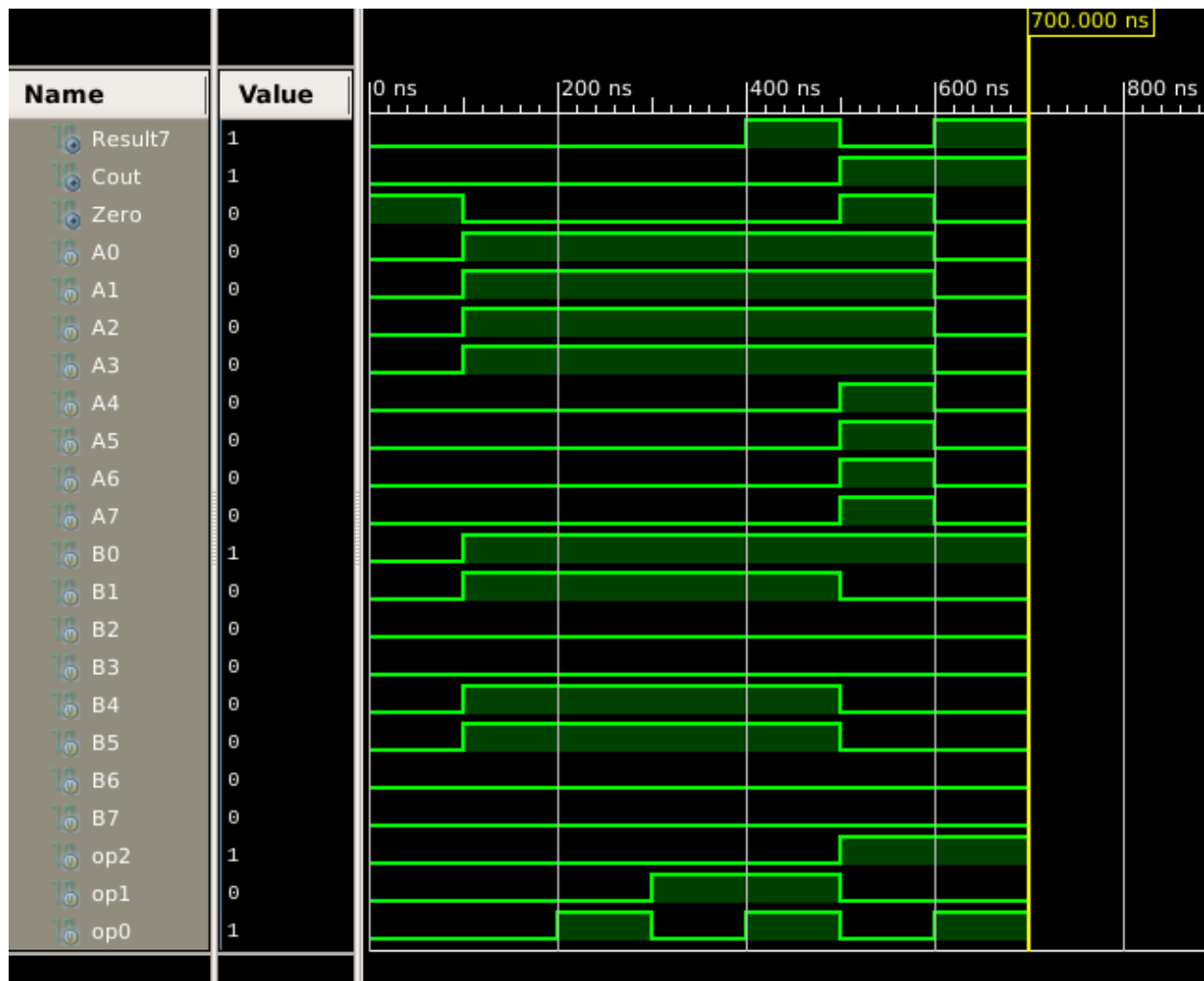


Figure 16(b): Simulation Waveform of the 8-bit ALU – Input and Opcode Signals

8-bit ALU Table:

Opcode	Operation	A	B	Result	Cout	Zero
000	AND	00001111	00110011	00000011	0	0
001	OR	00001111	00110011	00111111	0	0
010	XOR	00001111	00110011	00111100	0	0
011	NOT A	00001111	—	11110000	0	0
100	ADD	11111111	00000001	00000000	1	1
101	SUBTRACT	00000000	00000001	11111111	1	0

Table: Observation of Final 8-bit ALU

7. Result

The 8-bit ALU was successfully designed and simulated using Verilog HDL in Xilinx ISE 14.7. The individual 1-bit operations—AND, OR, XOR, NOT, addition, and subtraction—were tested first and produced the expected outputs. These operations were then combined into a single 1-bit ALU using opcode inputs, and eight 1-bit ALU blocks were connected to build the final 8-bit ALU. The final simulation also showed correct Result, Cout, and Zero outputs for the tested operations. Therefore, the objective of designing and verifying the 8-bit ALU was achieved.

8. GitHub Repository Commits

Commit: Add complete 8-bit ALU design, testbenches, and simulation results

Commit Link: <https://github.com/naima-tabassum/CSE450-VLSI-Lab/commit/4fe0701eaac2b51483b2aad4495c693c72c83fb0>

9. Conclusion

This experiment helped to understand how a complete ALU can be developed step by step from smaller logic blocks. The basic 1-bit operations were designed and tested first, and then they were combined using opcode inputs to form a 1-bit ALU. After that, eight 1-bit ALU blocks were connected to create the final 8-bit ALU. Through RTL schematics and simulation waveforms, the behavior of the logical and arithmetic operations was checked and verified. Overall, the experiment provided a clear practical understanding of ALU design, operation selection, carry/borrow handling, and the Zero output in Verilog.